

Implement Client Script & UI Policy (Incident)

Date: 2 October 2026
Team ID: SWTID-2026-5622
Project: Implement Client Script & UI Policy (Incident)
Domain: ServiceNow Developer Platform

SCENARIO Creating an Incident with UI Policy field control and Client Script validation	1 Load What happens when the Incident form opens?		2 Enter How does the user provide Incident data?		3 Control How do UI Policies shape the form dynamically?		4 Validate How do Client Scripts check input in real time?		5 Submit & Extend What happens once the record is submitted?			
	Steps What does the user typically do or experience at each stage?		Admin creates UI Policy High Impact Control on the Incident table (Active)	Open new Incident form; onLoad Client Script sets defaults	Fill in Caller, Short description and Category	Set Impact (1 - High) and Urgency	Condition checked: Impact is 1 - High	UI Policy Actions: fields shown, mandatory or read-only	onChange script checks and auto-populates values using g_form	onSubmit script blocks the save if rules are not met	Click Submit; record saved to the Incident table	Accurate data supports triage, routing and SLA
	Components & Interactions Which platform components are involved?		UI Policy record: Name, Table, Active	Incident Form / Workspace with onLoad Client Script	Caller, Short description and Category fields	Impact, Urgency and Priority fields	UI Policy Condition: Impact is 1 - High	UI Policy Actions: Visible, Mandatory, Read only	onChange and onSubmit Client Scripts	g_form getValue, setValue and error message	Submit action on the Incident form	Platform Database: Incident [incident] table
	Goals & objectives At each stage, what must the solution achieve?		Control fields without manual checks	Give users a clean, pre-filled form	Guide users to complete key fields	Capture accurate category and impact	Trigger only when Impact is High	Make required fields mandatory	Validate input in real time	Prevent submit when conditions are not met	Records meet defined business rules	Better data quality, reporting and SLA compliance
	Benefits & outcomes What works well once the controls are in place?		One reusable, clearly named policy	Defaults save time	Priority updates as Impact and Urgency change		Mandatory fields appear exactly when needed		Errors caught before the record is saved	Dependent fields fill in automatically	Complete, valid and consistent Incident data	
	Challenges & risks What goes wrong without these controls?		Relying on user awareness and manual checks		Key fields left blank or filled with wrong values		Too many visible fields confuse the user		Poorly written scripts can slow the form or conflict with UI Policies		Incomplete data hurts reporting and SLA compliance	
Areas of opportunity How could each stage be improved?		Use a Client Script only where a UI Policy cannot do the job		Add helpful field hints and guidance text		Extend the pattern to other conditions	Test on desktop and mobile	Keep validation messages short and clear	Test across supported browsers	Report on data quality over time		